

OMNINJA — Patch de Segurança do Sandbox (EC2)

O problema, exatamente como está no seu código hoje

`mini-services/sandbox-server/index.ts` :

- `server.listen(PORT, '0.0.0.0', ...)` → escuta em todas as interfaces, não só localhost
- Nenhuma rota (`/shell` , `/file/write` , `/file/read` , `/browser` , `/cleanup`) verifica quem está chamando
- CORS: `'Access-Control-Allow-Origin': '*'`
- `install-sandbox.sh` manda abrir a porta 3005 pra `0.0.0.0/0` no Security Group

Resultado: `/shell` roda `exec(cmd)` direto — qualquer POST de qualquer lugar do mundo executa comando arbitrário na sua máquina, com os privilégios de quem está rodando o processo. Também tem um path traversal pequeno em `/file/read` e `/file/write` (o código só remove a barra inicial do path, não impede `../../`).

Separado disso: `package.json` do `sandbox-server` tem `"start": "node index.js"` , mas o arquivo é `index.ts` — Node 20 (o que o install script instala) não roda `.ts` nativamente. Então o PM2 provavelmente está crash-loopando ou nunca subiu de verdade.

Verificação de comprometimento (fazer antes de tudo, é rápido)

Se essa porta ficou aberta por horas/dias, vale checar antes de simplesmente corrigir e seguir:

```
# processos estranhos
ps aux --sort=-%cpu | head -20
pm2 list

# cron jobs que você não criou
crontab -l
ls -la /etc/cron.d/

# chaves SSH que você não adicionou
cat ~/.ssh/authorized_keys

# logins recentes
last -20

# conexões de saída ativas (sinal de miner/botnet)
ss -tulpn
```

Se aparecer algo que você não reconhece, o mais seguro é não confiar mais nessa instância: suba uma EC2 nova, migre só o código (nunca o estado do sistema), e rotacione toda credencial que já existiu nessa máquina (chaves de API, `AUTH_SECRET`, credenciais AWS da própria instância).

1. Gerar o token de autenticação

```
openssl rand -hex 32
```

Guarde esse valor. Ele vai em **dois lugares**: no `.env` da EC2 (sandbox-server) e no `.env` do app Next.js — mesmo valor nos

dois.

```
# No .env da EC2 (mini-services/sandbox-server/.env)
SANDBOX_AUTH_TOKEN=cole_o_valor_gerado_aqui

# No .env do Next.js (onde já está o SANDBOX_URL)
SANDBOX_AUTH_TOKEN=cole_o_mesmo_valor_aqui
```

2. Patch em `mini-services/sandbox-server/index.ts`

Adicione logo abaixo de `const WORKSPACE_ROOT = ...`:

```
const SANDBOX_AUTH_TOKEN =
process.env.SANDBOX_AUTH_TOKEN;

function isAuthorized(req: IncomingMessage): boolean {
  if (!SANDBOX_AUTH_TOKEN) return false; // sem token
  configurado = nada roda (fail closed)
  return req.headers['authorization'] === `Bearer
  ${SANDBOX_AUTH_TOKEN}`;
}

function safeJoin(root: string, userPath: string):
string {
  const resolved = join(root, userPath.replace(/^\/+/,
  ''));
  if (!resolved.startsWith(root)) throw new
  Error('path traversal bloqueado');
  return resolved;
}
```

E logo depois do bloco `if (req.method === 'OPTIONS') { ... }`, antes do health check, adicione:

```
if (url !== '/health' && !isAuthorized(req)) {
  res.writeHead(401, { 'content-type':
```

```
'application/json' });  
res.end(JSON.stringify({ error: 'unauthorized' }));  
return;  
}
```

Isso deixa `/health` público (útil pra monitoramento) e exige o token em tudo que executa código, lê/escreve arquivo ou controla o navegador.

Troque as duas linhas que usam `join(workspace, path.replace(/^\//, ''))` (em `fileWrite` e `fileRead`) por `safeJoin(workspace, path)`.

Por fim, troque o bind de rede — escolha uma das duas:

```
// Se o Next.js roda NA MESMA instância EC2 (mais  
simples e mais seguro):  
server.listen(PORT, '127.0.0.1', () => { ... });  
  
// Se o Next.js roda em outro lugar (ex: Vercel) e  
precisa alcançar a EC2 por IP público:  
server.listen(PORT, '0.0.0.0', () => { ... }); //  
mantém, mas depende do passo 4 abaixo
```

3. Substituir `src/lib/sandbox-client.ts` inteiro por este

```
// OmniNinja – Sandbox Client (chama a EC2 Ubuntu  
remota)  
const SANDBOX_URL = process.env.SANDBOX_URL;  
const SANDBOX_AUTH_TOKEN =  
process.env.SANDBOX_AUTH_TOKEN;  
  
export const hasSandbox = !!SANDBOX_URL &&  
!!SANDBOX_AUTH_TOKEN;
```

```

function headers() {
  return {
    'content-type': 'application/json',
    ...(SANDBOX_AUTH_TOKEN ? { authorization: `Bearer
${SANDBOX_AUTH_TOKEN}` } : {}),
  };
}

async function call(path: string, body: unknown) {
  if (!SANDBOX_URL) throw new Error('SANDBOX_URL not
configured');
  const res = await fetch(`${SANDBOX_URL}${path}`, {
    method: 'POST',
    headers: headers(),
    body: JSON.stringify(body),
  });
  return res.json();
}

export const sandboxShell = (taskId: string, cmd:
string) => call('/shell', { taskId, cmd });
export const sandboxFileWrite = (taskId: string, path:
string, content: string) => call('/file/write', {
taskId, path, content });
export const sandboxFileList = (taskId: string) =>
call('/file/list', { taskId });
export const sandboxFileStrReplace = (taskId: string,
path: string, oldStr: string, newStr: string) =>
  call('/file/str-replace', { taskId, path, oldStr,
newStr });
export const sandboxBrowser = (action: string, args:
any, pageState?: { url?: string }) =>
  call('/browser', { action, args, pageState });

export async function sandboxFileRead(taskId: string,
path: string) {
  const data = await call('/file/read', { taskId, path
});
  return data.content;
}

export async function sandboxCleanup(taskId: string) {

```

```
if (!SANDBOX_URL) return;
await fetch(`${SANDBOX_URL}/cleanup`, { method:
'POST', headers: headers(), body: JSON.stringify({
taskId }) }).catch(() => {});
}

export async function sandboxHealth():
Promise<boolean> {
  if (!SANDBOX_URL) return false;
  try {
    const res = await fetch(`${SANDBOX_URL}/health`, {
signal: AbortSignal.timeout(5000) });
    return res.ok;
  } catch {
    return false;
  }
}
```

4. Corrigir o Security Group na AWS

```
# Achar o Security Group ID da instância (troque o IP
público real)
aws ec2 describe-instances \
  --filters "Name=ip-
address,Values=SEU_IP_PUBLICO_AQUI" \
  --query
"Reservations[].Instances[].SecurityGroups[].GroupId"
--output text

# Remover a regra aberta pra internet inteira na porta
3005
aws ec2 revoke-security-group-ingress \
  --group-id sg-XXXXXXX \
  --protocol tcp --port 3005 --cidr 0.0.0.0/0
```

- **Se o Next.js roda na mesma EC2:** pare por aqui — com o bind em `127.0.0.1` do passo 2, a porta 3005 nem precisa de regra

nenhuma no Security Group, ela não é mais alcançável de fora.

- **Se o Next.js roda em outro servidor com IP fixo:** adicione só esse IP —

```
aws ec2 authorize-security-group-ingress \
  --group-id sg-XXXXXXX \
  --protocol tcp --port 3005 --cidr
SEU_IP_DO_BACKEND/32
```

(Se o Next.js for pra Vercel/serverless no futuro, IP fixo não existe nesse caso — a única proteção real vai ser o token, o que já está feito no passo 1-3.)

5. Corrigir o start do PM2 (o bug do `.ts / .js`)

Na EC2:

```
# instalar Bun, se ainda não tiver
curl -fsSL https://bun.sh/install | bash
source ~/.bashrc

cd ~/omninja/mini-services/sandbox-server
pm2 delete omninja-sandbox 2>/dev/null || true
pm2 start bun --name omninja-sandbox --interpreter
none -- run index.ts
pm2 save

# testar
curl http://127.0.0.1:3005/health
curl -X POST http://127.0.0.1:3005/shell \
  -H "content-type: application/json" \
  -H "authorization: Bearer SEU_TOKEN_AQUI" \
  -d '{"taskId":"teste","cmd":"echo funcionando &&
whoami"}'
```

Se o segundo `curl` (sem o header de autorização) retornar `401`, o patch está correto.